Robotic Car Painting Cell - Simulation Report

Lassaad Mahmoudi | ENIG | assaadmahmoudi0@gmail.com
Software: ROS 2 Jazzy, MoveIt 2 (OMPL, KDL, FCL), Gazebo Harmonic (gz_ros2_control) | **Robot:** KUKA KR 210 L150 on a rail and cross slide
GitHub: github.com/AssaadMH/robotic-painting-cell | **Drive:** drive.google.com/drive/folders/1qovta20oiRVogkL21aGCfzKGMFycxMRL

> A KUKA KR 210 L150 paints a Togg T10X body, scaled to the 4,800 x 1,900 x 1,600 mm envelope of the brief, on a turntable in a Gazebo paint booth. The spray paths are cast onto the real body mesh, every motion is planned with MoveIt 2 against a full collision scene, and the recorded plan is executed in Gazebo by ros2_control. **94 of 94 strokes planned, 3,137 recorded robot states re-checked with 0 collisions**, nozzle tracking error 0.41 mm on average, paint coverage 98.4 - 100 %.

1. Software used

- **ROS 2 Jazzy** on Ubuntu 24.04 (WSL 2) and **MoveIt 2**: OMPL (RRTConnect) for joint-space moves, the Cartesian-path service for approach, paint and retreat, KDL inverse kinematics, FCL collision checking on the planning scene.
- **Gazebo Harmonic**: the paint booth world, the robot under **gz_ros2_control** with a `joint_trajectory_controller` over all 8 axes, and a camera sensor that films the demo. A player node sends the recorded MoveIt plan to that controller, turns the turntable, places the spray cone at the nozzle and recolours the paint as the gun passes.
- Python 3 (NumPy, trimesh) for the car preparation, ray casting, path generation and the video; everything is scripted and reproducible (`tools/`).

2. Robot selected

KUKA KR 210 L150 (six axes, spherical wrist, about 3.1 m reach), using the ROS-Industrial URDF and meshes (Apache-2.0). A UR20 (1.75 m reach) was ruled out on geometry: from a rail beside a 1.9 m wide car the roof centre line is about 2.2 m from the base axis. A long-reach industrial arm is the right class of robot for a car body. A 0.40 m **paint gun** with a 250 mm spray fan is the end effector; its nozzle tip is the TCP (`paint_tcp`).

3. Cell layout

An 11 x 8 m booth; the car stands on a turntable flush with the floor and is indexed by 180 degrees between the two halves. The robot sits on a pedestal on an 8 m rail platform along the car; a 0.9 m **cross slide** moves the base towards or away from the car, giving **eight axes**: rail, cross slide and six joints.



Fig. 1 - The cell in Gazebo Harmonic: booth, turntable, rail and cross slide, KUKA KR 210 L150, Togg body in primer.

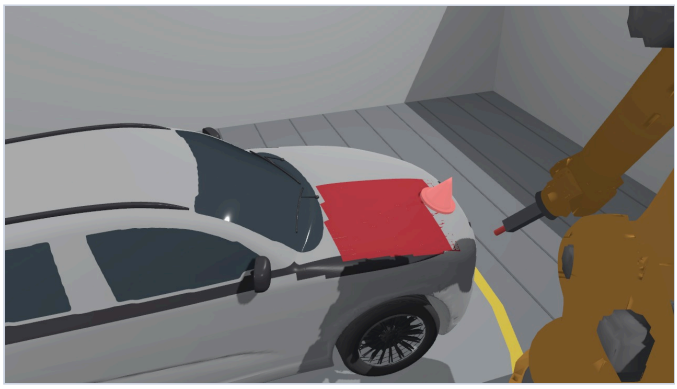


Fig. 2 - Hood stroke with the cross slide in (base 2.2 m from the car axis); the cone is the spray fan, red = painted.

Why a cross slide (reach study)

With the base fixed 2.2 m from the car, IK failed for every side-panel stroke even without collision checking. A MoveIt sweep (IK with a horizontal gun, no collision check) showed that no fixed base distance serves both the sides and the top:

Base distance from car axis	side panels reachable	roof reachable	hood reachable
2.4 m	0 / 8	5 / 8	8 / 8
2.6 m	3 / 8	5 / 8	8 / 8
2.8 m	7 / 8	5 / 8	6 / 8
3.0 m	8 / 8	2 / 8	4 / 8

The planner therefore picks the slide per stroke: **in (2.2 m) for the hood and roof, out (3.0 m) for the sides**. In the final plan 42 strokes use the slide in (0 - 0.15 m) and 52 out (0.8 - 0.9 m).

4. The car

The car is a downloaded **Togg T10X** mesh (460 k triangles, y-up, an airport "follow-me" variant). A script (`tools/prep_car.py`) splits it into its 8,544 connected parts and classifies them: the **body shell** (painted), wheels, trim, a roof lining, and parts that a body in the paint shop does not carry, which are removed: the **roof light bar**, the **livery decals** and the **number plates** (found as thin plates lying on the body). The shell includes the windows; they are found by casting a ray inwards from each face: rays that pass through an opening of the roof lining belong to glass. The car is then oriented (front +x), put on the turntable and scaled to the brief: **4.80 x 1.90 x 1.60 m** (body width; mirrors 2.12 m).

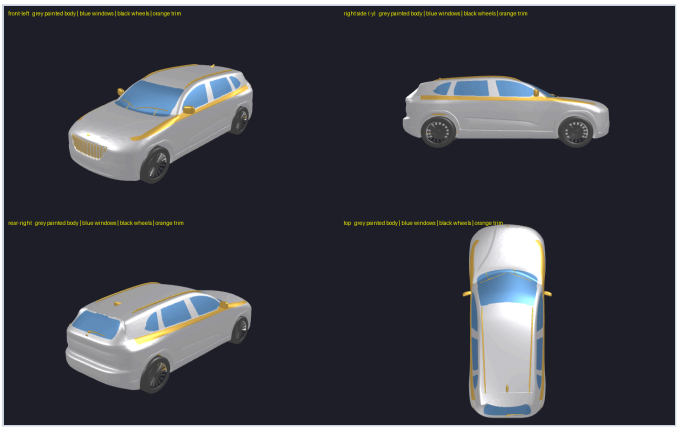


Fig. 3 - Prepared car: painted panels (grey), windows (blue, masked), wheels, trim (orange). Light bar, livery and plates removed.

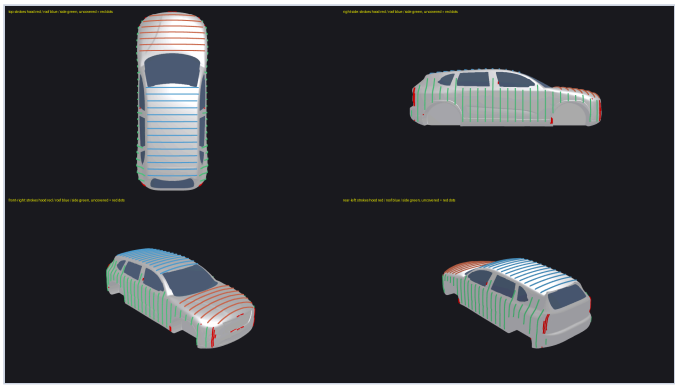


Fig. 4 - The 94 strokes cast onto the body: hood (red), roof (blue), side panels (green). Red dots = surface not covered.

5. Approach and trajectory / path planning

5.1 Painting paths from the body mesh

The generator (`tools/gen_paths.py`) **casts rays onto the body** for exact points and smooth (interpolated) normals and lays a zig-zag raster: stroke pitch = fan width x (1 - 30 % overlap) = 175 mm, spaced by **arc length** along the profile, gun on the surface normal at a **250 mm standoff**. Panel limits come from the mesh: the hood ends at the windshield, the roof lies between the windshield and the rear window, the sides run from the sill to the window line. Wheels block the rays; window openings let them through, and a ray that slips through an opening (landing on the floor pan) cuts the stroke. On the inward-sloping lower doors the gun is kept level.

Surface	strokes	path	coverage
Hood (both halves)	14	11.2 m	99.3 %
Roof (both halves)	28	17.0 m	100 %
Side panels, sill to window line (2 x 26)	52	28.0 m	98.4 %
Total	**94**	**56.2 m**	

Coverage = share of surface sample points (2 cm grid) within half a fan width of a stroke.

5.2 Motion planning per stroke (MoveIt 2)

1. **Rail / cross slide** to the stroke's station (collision-checked, arm folded first if needed).
2. **Pre-stroke pose** 200 mm beyond the standoff: KDL IK from the current posture and several elbow-up seeds, then an **OMPL RRTConnect** move.
3. **Straight approach** to the standoff, then the **painting pass** along the surface polyline (Cartesian path, 20 mm interpolation, collision checked at every point), re-timed to a 0.40 m/s nozzle speed; then a **retreat** along the normal.

If a stroke fails, the planner retries from other rail stations, other cross-slide positions and with the gun tilted by 12 degrees: **83 of 94 strokes succeeded at the first attempt, 11 with a fallback, none failed**. Between the two halves the arm parks and the car turns.

6. Collision checking

The planning scene contains the robot (rail, carriage, slide, pedestal, arm, gun), the **car** (body, glass, wheels, mirrors and trim, 40 k triangles, re-posed for each turntable angle), the floor and the booth walls. Collisions are checked at five levels: (1) IK with `avoid_collisions`; (2) OMPL state validity; (3) every interpolated point of every Cartesian pass; (4) the turntable rotation, swept in 5-degree steps; (5) an independent pass that re-checks every recorded trajectory state.

Check	Result
Independent re-check of the recorded plan	3,137 states, 0 collisions
Turntable sweep, 5-degree steps	36 / 36 positions collision free
Negative test: hood stroke with the nozzle 4 cm *inside* the body	rejected: the collision-aware planner stops at 70 % of the pass; the unconstrained path has 39 of 45 states in collision, first contact gun - car
Same stroke at the 250 mm standoff	100 % planned, 0 of 42 states in collision

The checks shaped the result: strokes first ran up the window pillars and failed near the mirrors and roof rails (5 of 94); the side strokes now stop at the window line, and the lower-door points use a level gun instead of aiming up from the floor.

7. Execution in Gazebo

The recorded plan (7,807 trajectory points over the 8 axes) is sent as one `FollowJointTrajectory` action goal to the joint_trajectory_controller running in Gazebo through gz_ros2_control (position interface). The PC renders in software (no supported GPU in WSL, about 3.5 s per image), so the world is paused and stepped frame by frame: per camera frame (0.5 s of simulated time) **ten 0.05 s physics steps and ten controller periods**. A player node places the spray cone on the nozzle, recolours the paint strips the gun has passed (1,396 strips) and moves the camera. These follow the arm's *measured* position along the plan, predicted one frame ahead because the cone must be placed before the step is simulated; the cone is within 5 cm of the real nozzle in 83 % of the paint frames. The demo video plays the frames as a x5 time-lapse.

Comparing the joint states that Gazebo publishes with the plan exposed three integration problems, all fixed:

- The controller rejected the trajectory when it was stamped with its future start time, and the single 7,807-point topic message was never delivered. It is now sent unstamped through the action interface on the frame where the job starts.
- With one 0.5 s physics step per frame, the coupled wrist dynamics diverged whenever A4 and A6 turned quickly near A5 = 0: A5 was thrown to its 125-degree limit and the nozzle left the path by up to 1.15 m in 15 of 130 paint frames. Ten 0.05 s steps per frame removed this. Physics costs about 0.01 s per frame, so the render time did not change.
- The arm does not run exactly on the show clock (controller start, position lag), so the cone, paint and camera use the measured phase, not a fixed delay.

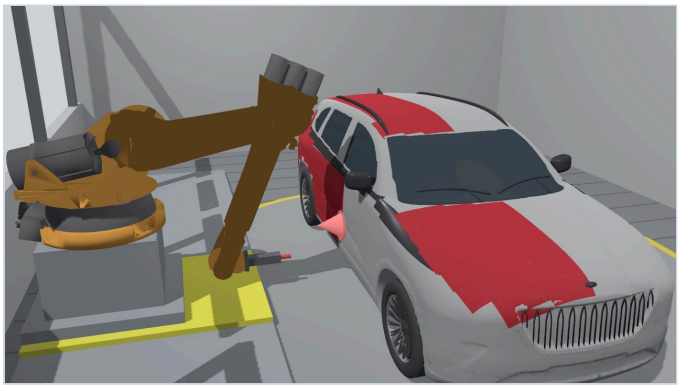


Fig. 5 - Side panel: cross slide out, vertical strokes, the rail follows the car.



Fig. 6 - Finished car after both halves and the 180-degree turn.

8. Results

Metric	Result
Strokes planned and executed	**94 / 94** (11 with a fallback, 0 failed); planning time 138 s
Collision re-check of all recorded motion	**3,137 states, 0 collisions**
Paint coverage	hood 99.3 %, roof 100 %, side panels 98.4 % (sill to window line)
Nozzle tracking error (forward kinematics vs planned path)	mean 0.41 mm, max 2.8 mm over 94 passes
Nozzle speed on paint passes	mean 0.26 m/s (target 0.40; short strokes are dominated by acceleration)
Simulated cycle time, whole car	12.7 min: 4.6 min spraying, 8.1 min rail / joint repositioning
Gazebo execution (ros2_control)	1,866 frames; nozzle distance from the planned path on paint passes: mean 3.0 mm, max 207.8 mm (all job frames: max 304.3 mm); 1,396 / 1,396 paint strips recoloured

9. Limitations and next steps

- Not painted: front and rear fascias, the window pillars and the sill under-curve (they need dedicated short programs and gun orientations).
- Paint is shown as recoloured strips over the body; there is no film-thickness or overspray model. The Gazebo run replays a kinematically planned motion; robot dynamics and controller tuning are not studied.
- Software rendering limits the Gazebo camera to about 3 s per frame, hence the time-lapse. KDL is a local IK solver (hence the multi-seed retries). Cycle time could drop by overlapping rail and arm moves and ordering strokes to reduce repositioning.

Credits: KUKA KR 210 URDF and meshes from ROS-Industrial `kuka_experimental` (Apache-2.0); Togg T10X mesh: third-party download, used for this simulation only (light bar, livery and plates removed, scaled). Everything else (cell, car preparation, paths, planner, Gazebo world and player, video) written for this assessment.